

Chapter3

1.

给定一个栈S，使用一个辅助队列Q来实现

使用C++语言简要实现

```
void reverse_top_k(std::stack<int>& S, int k) {
    std::queue<int> Q;
    int count = 0;

    // 将前 k 个元素出栈并入队列
    while (count < k && !empty(S)) {
        enqueue(Q, pop(S));
        count += 1;
    }

    // 将队列中的元素出队并重新压回栈中
    while (!isEmpty(Q)) {
        push(S, dequeue(Q));
    }
}
```

```
bool is_palindrome(std::stack<int>& S) {
    std::queue<int> Q;
    bool is_palindrome = true;

    //将栈中的元素依次出栈并入队列，同时恢复栈的原始顺序
    while (!empty(S)) {
        int temp = pop(S);
        enqueue(Q, temp);
        push(S, temp);
    }

    // 比较栈与队列中的元素，判断是否为回文
    while (!isEmpty(Q)) {
        if (pop(S) != dequeue(Q)) {
            is_palindrome = false;
            break;
        }
    }

    // 将队列中的元素重新压回栈，恢复栈的原始顺序
    while (!isEmpty(Q)) {
        push(S, dequeue(Q));
    }

    return is_palindrome;
}
```

2.

似乎有问题

3.

证明：从初始输入序列 $1, 2, \dots,$ ，可以利用一个栈得到输出序列 $p_1, p_2, \dots, p_n$ ($p_1, p_2, \dots, p_n$ 是 $1, 2, \dots,$ 的一种排列)的充分必要条件是：输出序列中不存在下标 i ,

必要性

假设存在下标 i, j, k ，使得 $i < j < k$ 且 $p_i > p_k > p_j$ ，根据栈的后劲先出特性，较大的数字 p_i 会在较小的数字 p_j 之后出栈，那么若 p_k 在 p_i 之后出栈，则在 p_k 出栈是，栈中一定

充分性

假设输出序列中不存在下标 i, j, k ，使得 $i < j < k$ 且 $p_i > p_k > p_j$ 可以通过栈操作实现该输出序列，从输入序列中主格读取元素，若栈为空或栈顶元素小于当前元素，就将当前元素入栈，若栈顶元素 $\geq$ 当前元素，则将栈顶元素出栈并作为输出，循环次过程直到满足上一个条件，最后将当前元素入栈接口。

因此, 从初始输入序列 $1, 2, \dots,$ ，可以利用一个栈得到输出序列 $p_1, p_2, \dots, p_n$ ($p_1, p_2, \dots, p_n$ 是 $1, 2, \dots,$ 的一种排列)的充分必要条件是：输出序列中不存在下标 i, j, k ，使